

An Adjective Syntax for Concepts

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Motivation](#)
4. [Details](#)
5. [What about auto?](#)
6. [And what about lambdas?](#)
7. [Recap](#)
8. [Future directions](#)
9. [Alternatives](#)
10. [Questions](#)
11. [Acknowledgements](#)

Revision history

- P0807r0: Initial proposal.

Summary

We propose to change the *constrained-parameter* syntax for concepts to be more general, more explicit and more expressive by using the grammatical structure of “*attributive adjective, subject noun, predicate noun*”:

template <Sortable	typename	T	> void f(T& collection);
template <Even	int	N	> void g(Foo (&pairs)[N]);
<i>attr. adjective</i>	<i>subj. noun</i>	<i>pred. noun</i>	

This idea is not original; to my best knowledge it was first proposed by Richard Smith in Issaquah 2016, and I have since heard it restated as Richard’s idea by several others, including members of the BSI and on the std-proposals mailing list.

Motivation

In the present C++ working paper ([N4687](#)), the “Concepts” feature consists of the core mechanics of template constraints ([temp.constr]), as well as a “convenience syntax” in the form of *constrained-parameters*, which offer a short-hand access to a limited subset of constraints. Specifically, a constrained template like `template <Sortable T>` is syntactic sugar for `template <typename T> requires Sortable<T>` when `Sortable` is a type concept, and more generally the kind of `T` is deduced from the prototype parameter of the concept.

This syntax is superficially convenient, but it suffers from the problem that it creates an ambiguity: Does `template <Foo X>` have an unconstrained non-type parameter or a constrained (type or template) parameter? This ambiguity may not be an immediate concern, but it presents a kind of dead end for the design space: Additional syntactic shorthands, such as the terse function syntax from the Concepts TS, bring their own set of new ambiguities. (For example, is `f(Foo x, Foo y)` a function or a function template? Does it have one or two template parameters?)

We propose to improve the situation by revising the *constrained-parameter* syntax. (The core Concepts engine will remain unaffected.) The proposed changes establish a consistent set of syntactical rules and markers that resolve the present ambiguities and will also work well with future additional syntactic sugar for function and variable declarations.

Details

There are three *kinds* of entities in C++ that one can parametrise over:

Kind	Disambiguator	Declaration example	Template parameter example
1. things that have a type	nothing	<code>int a = x;</code> (x is a value)	<code>template <int N></code>
2. things that are a type	<code>typename</code>	<code>using T = typename foo::x;</code> (x is a type)	<code>template <typename T></code>
3. templates	<code>template</code>	<code>using A = typename foo::template x<T></code> (x is a template)	<code>template <template </* ... */> typename Tmpl></code>

Templates can be specialised to form all three of these kinds (variable/function templates for (1), class templates for (2), and alias templates for (3)). Let us call the three kinds *values*, *types* and *templates* for short, just for the sake of brevity and for the scope of this proposal (so functions are values for now). When we need to be explicit about the kind of an entity, either because a name is dependent, or because we are specifying template parameters, we distinguish the three kinds with the *disambiguator* shown in the table above.

If we examine the template parameter grammar in C++ from the perspective of natural language, an analogy suggests itself: In `int N`, `typename T`, `template ... Tmpl`, the name of the parameter that is being declared is naturally a predicate noun, and the word that precedes it is the subject: “The integer is `N`.”, “A type called `T`.”, etc. The disambiguator is a noun making up that subject, and for the *value* case, the actual type name of the parameter is the noun. Where does this leave concept constraints? Constraints constrain something, so they are a kind of qualification, an attribute. The simplest kind of attribute is the attributive adjective: A *red* door, an *even* number. Constraint names can often be read as adjectives (or perhaps as some more complex attributive).

This little excursion into natural language suggests a modification to the C++ grammar that makes the language more expressive and that resolves the current problems. The solution is to retain *both* the attributive adjective *and* the subject noun in the parameter declaration:

1. `template <Even int N> for template <int N> requires Even<N>` (unary non-type constraint).
2. `template <Sortable typename T> for template <typename T> requires Sortable<T>` (unary type constraint).
3. `template <Rebindable template <typename> typename Tmpl> for template <typename> typename Tmpl> requires Rebindable<Tmpl>` (unary template constraint); but see below.

Both the adjective and the predicate are optional, of course: A parameter can be unconstrained and need not be named. But the general template parameter now consists of three parts, and there is now no ambiguity: If `typename` is present, it is a type parameter, if `template` is present, it is a template parameter, and if neither is present, it is a non-type parameter.

A concept that can be used as an adjective in this way is required to be *unary*, i.e. to constrain only one template parameter, and it has to have the appropriate kind of prototype parameter: value concepts must have a non-type parameter of the correct type, type concepts must have a type parameter, and template concepts must have a template parameter of a compatible signature (allowing for `...` to match non-variadic templates, etc.). Additional template parameters are allowed, just as they are in the status quo, and are filled in after the first parameter: `template <VeryEven<A, B, C> int N>` becomes `template <int N> requires VeryEven<N, A, B, C>`.

For constrained template template parameters, we may wish to use the simpler form `template <Rebindable template Tmpl>` and leave the template signature entirely to be determined by the concept. On the other hand, using the full template signature allows the user to request a template of a specific signature that is *also* constrained by a (possibly more generic) concept. We could also allow both a short form (without `typename` and `class`) that deduces the template signature from the concept, as well as a long form (with `typename` or `class`). If the long form is used, the user-provided signature must be compatible with the concept's prototype parameter.

What about `auto`?

An `auto` parameter is a non-type parameter whose type is deduced. As such, we may wish to constrain both its value and also its permissible types.

The most conservative solution is to continue the same logic as above, and require the concept to have an exactly matching prototype parameter, namely an `auto` parameter. For example, `template <EvenInteger auto N>` becomes `template <auto N> requires EvenInteger<N>`, and the concept could be something like:

```
template <auto N>
    concept EvenInteger = (std::is_integer_v<decltype(N)>) && (N % 2 ==
0);
```

However, we can imagine different directions or generalisations:

- Allow type concepts to constrain `auto` parameters: `template <Foo auto N>` becomes `template <auto N> requires Foo<decltype(N)>` when `Foo` is a type concept (“automatic `decltype` unwrapping”).
- `template <Even auto N>` might be allowed but a hard error or SFINAE condition when `N` is deduced to anything other than `int`.

Let us call a concept whose prototype parameter is `auto` an *auto* concept. Naturally, `auto` concepts constrain `auto` parameters. But we may also allow `auto` concepts to constrain typed non-type parameters: `template <EvenInteger long N>` becomes `template <long N> requires EvenInteger<N>` and deduces the type of `N` as `long`. The adjective syntax allows us to write specific templates (e.g. using `long`) constrained by generic, reusable concepts (e.g. `EvenInteger`).

And what about lambdas?

A C++14-style generic lambda contains a function template that does not use a template introducer, and instead declares a parameter with type specifier `auto`, where `(auto x)` stands for a function template of the form `template <typename T> (T x)`. To allow

constrained generic lambdas, only type constraints may be applied to the (implied) template parameter. The natural syntax that suggests itself here is to perform “`decltype` unwrapping” and admit type-constraining concepts of the form

```
[](Sortable auto x) { /* ... */ }
```

to stand for

```
[]<Sortable typename T>(T x) { /* ... */ }  
// which is, behind the scenes:  
template <Sortable typename T> auto operator()(T x) const { /* ... */ }  
// or in full:  
template <typename T> requires Sortable<T> auto operator()(T x) const {  
/* ... */ }
```

Recap

☒ hide *params/args* examples

Constraint kind	Concept example	Example usage
Unary non-type template constraint	<pre>template <int N> concept NonTypeFoo = /* ... */;</pre>	<pre>template <NonTypeFoo int V> struct X; // requires NonTypeFoo<V></pre>
Unary type template constraint	<pre>template <typename T> concept TypeBar = /* ... */;</pre>	<pre>template <TypeBar typename T> struct Y; // requires TypeBar<T></pre>
Unary template template constraint	<pre>template <template <typename, int, typename...> typename Tmpl> concept TemplateQuz = /* ...</pre>	<pre>template <TemplateQuz template < typename, int, bool, char> typename</pre>

Constraint kind	Concept example	Example usage
	<pre>*/;</pre>	<pre>Tmpl> struct Z; // long form // requires TemplateQuz<Tmpl> template <TemplateQuz template Tmpl> struct Z; // short form, Tmpl is <typename, int, typename...> // requires TemplateQuz<Tmpl></pre>
Unary auto template constraints • on auto non-type parameters • on typed non-type parameters	<pre>template <auto N> concept EvenInteger = std::is_integer_v<decltype(N)> && (N % 2 == 0);</pre>	<pre>template <EvenInteger auto N> struct W1; // requires EvenInteger<N> template <EvenInteger std::size_t N> struct W2; // requires EvenInteger<N>, deducing std::size_t</pre>
Unary type and non-type constraints on auto parameters • decltype-unwrapping • value-constraining	<pre>template <typename T> concept Integral = std::is_integer_v<T>; template <int N> concept Even = (N % 2 == 0);</pre>	<pre>template <Integral auto N> struct W3; // requires Integral<decltype(N)> template <Even auto N> struct W4; // requires Even<N>,</pre>

Constraint kind	Concept example	Example usage
		and decltype(N) must be int
Function template constraints on generic lambdas (type constraints only)	<pre> [] (TypeBar auto x) equivalent to: [] <TypeBar typename T> (T x) equivalent to: [] <typename T> requires TypeBar<T> (T x) (hypothetical) </pre>	<pre> // // // </pre>

The changes in a nutshell:

- The *constrained-parameter* syntax for templates becomes more verbose, more explicit and more general.
- Template parameters become (again) unambiguously decorated by their familiar disambiguator as non-type, type and template kind.
- In particular, constrained `auto` parameters are decorated with an explicit `auto` and can additionally be constrained with type concepts (via “`decltype` unwrapping”) and with value concepts (via required type matching).
- Constrained template template parameters and constrained value parameters may be more specialised than their concept’s prototype parameter.
- The proposed style extends naturally to generic lambdas and other kinds of syntactic shorthands. It provides an unambiguous syntactic marker (`auto`) that a template is being declared, and that each declared constrained parameter is a distinct template parameter.

Future directions

Function templates

We [already demonstrated](#) how the adjective syntax may be reused to allow constrained generic lambdas. Since lambdas share many characteristics with ordinary functions, it is natural to allow ordinary function templates to use the same parameter declaration syntax as generic lambdas. Putting the constraint in attributive position leaves the `auto` keyword as an unmistakable signifier that the declaration is a function template. (The alternative syntactic shorthand that was present in the Concepts TS omits the `auto` keyword when a constraint is present, which leaves it unclear at a glance whether a function or a function template is being declared.)

```
void f(Foo auto x, Bar auto y);

// short for:

template <typename T, typename U> requires Foo<T> && Bar<U>
    f(T x, U y);
```

A side note: the syntax in the Concepts TS left it visually unclear whether a repeated constraint in the parameter list refers to one single or several distinct template parameters, e.g. whether `void f(Foo x, Foo y)` is `template <typename T> void foo(T, T)` or `template <typename T1, typename T2> void foo(T1, T2)`. (The TS does have a definite rule, but the point is that a reader needs to know and remember (or look up) that rule.) With the proposed adjective style, the function template might be spelled `void f(Foo auto x, Foo auto y)`, which, by analogy with existing uses of `auto` in C++17, makes it reasonably obvious that two distinct template parameters are being declared.

Variables and return types

Another kind of type that may be decorated with constraints is the type of a variable or the return type of a function. At present, `auto` is allowed for both; adding a constraint attribute may conceivably be useful. Moreover, deduction could be allowed for template parameters.

```
template <typename T> void f(T t) {
    Sortable auto x = t.GetAll(); // ill-formed unless "requires
    Sortable<decltype(x)>"
    std::vector<Desirable typename> v = t.RetrieveEvery(); // not
    "Desirable auto"; type, not value
}

Sortable auto g(auto y) { // maybe ill-formed, maybe SFINAE-friendly
    return y.GetAll();
}
```

Multiple concepts

Finally, we might consider allowing more than one concept name to appear before the noun, interpreted as “and”, as in `template <Sortable Movable typename C>`. The presence of the noun (here `typename`) makes it clear that `Sortable` and `Movable` are concepts.

Alternatives

The obvious alternative to the proposed change is to retain the status quo. The current *constrained-parameter* syntax is shorter. Type parameters occur much more often than value and template parameters, and so the loss of information about the parameter kind may be an acceptable trade-off. (After all, the purpose of syntactic sugar is to make common constructions convenient.)

We see the value of this proposal only partly in its increased generality and explicitness. The other part lies in the future directions for other kinds of syntactic shorthands. Reusing the syntax of the status quo is problematic, since for even shorter kinds of abbreviations it is prone to ambiguities. By contrast, this proposal offers a simple principle by which the keywords `auto`, `typename`, and `template` are consistently present to signal that a kind of template argument deduction is in place. Additionally, they offer a natural place for constraints on those arguments.

Another alternative is to place the concept adjective in predicative rather than attributive position (*The door is red.* vs. *the red door*). This would perhaps require some additional punctuators, e.g. `typename T : Sortable`. This idea seems overly inventive, and even though it is not substantially different from the proposed attributive position, it would perhaps not play as nicely with future directions (e.g. `vector<Sortable typename>` vs. `vector<typename : Sortable>`), or `Sortable auto x` vs. `auto : Sortable x` (or even `auto x : Sortable`).

Questions

1. Do we want an “adjective” style for constrained parameters (i.e. the subject is mandatory)?
2. Where should the adjective go: Attributive (`Sortable typename T`) vs. predicative (`typename T : Sortable`)
3. Support for `auto` parameters: a) Only `auto` concepts, b) allow `decltype`-unwrapping use of type concepts, c) allow value concepts (subject to matching types), d) allow both?
4. Do we want constrained generic lambdas now, or defer to future proposals?

Acknowledgements

Many thanks to Tom Honermann for valuable feedback.